SIMATS ENGINEERING

Department of Computer Science & Engineering ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1-ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	I Raiyan	Reg.No.:	192521421
Department:	IT	Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I K . A R I S T O T L E (1 9 2 5 1 1 1 9 6) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ARISTOTLE.K

MARKS ALLOCATION

	Problem Understanding (2 0%)	**Method Selection(20%)**	**Solution Process(25%)**	**Accuracy of Calculation(20%)**	**Interpretation of Results(15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 1

A processor performs arithmetic operations on signed 8−bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Answer

1. Problem Understanding (20%)

The processor performs arithmetic on **8-bit signed integers** using the **2's complement representation**.

Given:

- **+45**
- **−23**

We need to:

1. Convert both numbers into 8-bit binary.
2. Perform **Addition (+45 + (−23))**.
3. Perform **Subtraction (+45 − (−23))**.
4. Show every intermediate step.
5. Check whether **overflow** occurs.
6. Explain how the processor detects overflow.
7. Interpret the final result.

2. Method Selection (20%)

The processor uses **2's complement arithmetic**.

Rules Used

- Positive numbers → Direct binary representation.
- Negative numbers → Find 2's complement.
- Binary addition is performed normally.
- Ignore the carry out of the MSB.
- Overflow occurs only when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

3. Binary Conversion

(a) Convert +45
Decimal:

45

Binary:

45 = 32 + 8 + 4 + 1

Binary (8-bit):

00101101

(b) Convert −23

Step 1:

23 in binary

00010111

Step 2:

1's Complement

11101000

Step 3:

Add 1

11101000

+00000001

11101001

Therefore,

−23 = 11101001

4. Addition

(+45) + (−23)

00101101

+ 11101001

00010110

(Carry out is

ignored.) Binary

Result:

00010110

Convert back to

decimal: 16 + 4 + 2

=22

Final Answer

45 + (−23) = 22

Overflow Check

Operands:

+45 → Positive

−23 → Negative

Adding numbers with different signs **cannot produce overflow**.

Overflow = No

5. Subtraction

(+45) − (−23)

Subtracting a negative number is equivalent to adding the positive

value. So,

45 − (−23)

=

45 + 23

Convert +23

00010111

Now add:

00101101

+00010111

01000100

Binary Result:

01000100

Decimal:

64 + 4

=68

Final Answer

$45 - (-23) = 68$

Overflow Check

Operands:
45 → Positive

23 → Positive

Maximum 8-bit signed value:

+127

Since

68 < 127

there is **no overflow**.

6. Overflow Detection

The processor checks the **carry into** and **carry out of the Most Significant Bit (MSB).**

Overflow occurs when:

$\text{Carry into MSB} \neq \text{Carry out of MSB}$

or equivalently,

- Positive + Positive = Negative ❌
- Negative + Negative = Positive ❌

For this problem:

Addition

Positive + Negative

No overflow.

Subtraction

Positive + Positive

Result = Positive.

No overflow.

7. Interpretation of Results (15%)

Addition

$+45 + (-23)$

= +22

Binary Result:

00010110

The processor correctly performs addition using **2's complement arithmetic**, and the result is within the valid range of

`−128 to +127`. Hence, **no overflow occurs**

Subtraction

`+45 − (−23)`
= +68

Binary Result:

01000100

Subtracting a negative number is equivalent to addition. The result is also within the valid 8-bit signed range, so **no overflow occurs**.

Final Answers

Operation	Binary Result	Decimal Result	Overflow
`+45 + (−23)`	`00010110`	22	No
`+45 − (−23)`	`01000100`	68	No

Conclusion

The processor uses **2's complement representation** to perform signed arithmetic. Both addition and subtraction are executed through binary addition, with negative numbers represented using 2's complement. Overflow is detected by comparing the carry into and carry out of the MSB (or by checking operand and result signs). In this case, **both operations produce correct results without overflow**, satisfying the requirements of signed 8-bit arithmetic.

Q2. Carry Look-Ahead Adder (CLA)

Question:
A high-performance computing system replaces a ripple carry adder with a 4-bit Carry Look-Ahead Adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify why CLA improves speed in modern processors.

Problem Understanding (20%)

A Ripple Carry Adder (RCA) adds binary numbers by passing the carry from one bit to the next. Since each carry depends on the previous carry, the total delay increases with the number of bits.

A Carry Look-Ahead Adder (CLA) overcomes this limitation by calculating all carry bits simultaneously using **Generate (G)** and **Propagate (P)** signals. This significantly reduces computation time and improves processor performance.

Method Selection (20%)

Use the **4-bit Carry Look-Ahead Adder (CLA)** method.

Given Inputs

A = $\mathbf{1011_2}$ (11_{10})

B = $\mathbf{0110_2}$ (6_{10})

Initial Carry,
$C_0 = 0$

Solution Process (25%)

Step 1: Write the Inputs

Bit	A	B
A_3 B_3	1	0
A_2 B_2	0	1
A_1 B_1	1	1
A_0 B_0	1	0

Step 2: Calculate Generate (G) and Propagate

(P) Formula:

Generate (G) = A × B

Propagate (P) = A ⊕ B

Bit	A	B	Generate (G)	Propagate (P)
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Step 3: Calculate Carry Bits

Carry equations:

$C_1 = G_0 + P_0C_0$

$= 0 + (1 \times 0)$

$=$ **0**

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$

$= 1 + (0 \times 0) + (0 \times 1 \times 0)$

$=$ **1**

$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$= 0 + (1 \times 1) + (1 \times 0 \times 0) + (1 \times 0 \times 1 \times 0)$
$=$ **1**

$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

$= 0 + (1\times0) + (1\times1\times1) + (1\times1\times0\times0) + (1\times1\times0\times1\times0)$

$= \mathbf{1}$

Step 4: Calculate Sum Bits

Formula:

$\mathbf{S = P \oplus Carry}$

$S_0 = P_0 \oplus C_0$

$= 1 \oplus 0 = \mathbf{1}$

$S_1 = P_1 \oplus C_1$

$= 0 \oplus 0 = \mathbf{0}$

$S_2 = P_2 \oplus C_2$

$= 1 \oplus 1 = \mathbf{0}$

$S_3 = P_3 \oplus C_3$

$= 1 \oplus 1 = \mathbf{0}$

Final Output

Carry = **1**

Sum = $\mathbf{0001_2}$

Therefore,

$\mathbf{1011_2 + 0110_2 = 10001_2}$

Decimal Verification:

11 + 6 = **17**

Binary = $\mathbf{10001_2}$ ✓

Accuracy of Calculation (20%)

· Generate signals calculated correctly. ·
Propagate signals calculated correctly. · Carry values computed using CLA equations.

· Sum bits verified with decimal conversion.

· Final Answer:

Sum = 10001_2 (17_{10})

Comparison: Ripple Carry Adder vs Carry Look-Ahead Adder

Feature	Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
Carry generation	Sequential	Parallel
Speed	Slower	Faster
Delay	High (carry ripples through all stages)	Low (carries computed simultaneously)
Hardware complexity	Simple	More complex
Performance	Suitable for small circuits	Suitable for high-speed processors

Interpretation of Results (15%)

The Carry Look-Ahead Adder improves processor speed by calculating carry signals in advance instead of waiting for each previous carry. This reduces propagation delay and enables faster arithmetic operations. Although the CLA requires additional hardware for the generate and propagate logic, its high-speed performance makes it ideal for modern CPUs, digital signal processors (DSPs), and high-performance computing systems where fast arithmetic operations are essential.

Question 3: Booth's Algorithm (−6 × +5)

The processor must multiply two signed binary numbers using **Booth's Algorithm**.

- Multiplicand (M) = **−6**
- Multiplier (Q) = **+5**
- Number of bits = **4**
- Initial Booth bit (Q_{-1}) = **0**

Booth's Algorithm efficiently performs signed multiplication by reducing the number of addition and subtraction operations.

Booth's Algorithm uses:

- **A (Accumulator)** = 0000
- **Q (Multiplier)** = 0101 (+5)
- **M (Multiplicand) = 1010 (−6)**
- **−M** = 0110 (+6)
- **Q_{-1} (Booth Bit)** = 0

Booth's Rules

Q_0 Q_{-1}	Operation
00	No operation
11	No operation
01	A = A + M
10	A = A − M (= A + (−M))

After every operation, perform **Arithmetic Right Shift (ARS)**.

Initial Values

Register Value

A 0000

Q 0101

Q_{-1} 0

M 1010 (−6)

−M 0110 (+6)

Cycle 1

Q_0Q_{-1} = **10**

Operation:

A = A − M

0000

+0110

0110

After Arithmetic Right Shift:

A = 0011

Q = 0010

Q_{-1} = 1

Cycle 2

Q_0Q_{-1} = **01**

Operation:

A = A + M

0011
+1010

1101

After Arithmetic Right

Shift: A = 1110

Q = 1001

Q_{-1} = 0

Cycle 3

Q_0Q_{-1} = **10**

Operation:

A = A − M

1110

+0110

0100

After Arithmetic Right

Shift: A = 0010

Q = 0100

Q_{-1} = 1

Cycle 4

Q_0Q_{-1} = **01**

Operation:

A = A + M

0010

+1010

1100

After Arithmetic Right

Shift: A = 1110

Q = 0010

Q_{-1} = 0

Final Product

Combine **A** and **Q**

A = 1110

Q = 0010

Result = 11100010

Convert from 2's complement:

$11100010_2 = -30_{10}$

Since

[
(-6)\times(+5)=-30

]

the multiplication is **correct**.

Accuracy of Calculation (20%)

- Multiplicand = −6 ✓
- Multiplier = +5 ✓
- Four Booth cycles completed ✓
- Arithmetic right shifts applied correctly ✓
- Final product = $11100010_2 = -30_{10}$ ✓

Interpretation of Results (15%)

- Booth's Algorithm correctly multiplies signed binary numbers without separately handling the sign.
- It reduces the number of addition and subtraction operations by examining pairs of multiplier bits (Q_0Q_{-1}), making multiplication more efficient than traditional shift-and-add methods.
- The algorithm is widely used in processors because it speeds up signed multiplication and reduces hardware complexity.

Final Answer

Product = $11100010_2 = -30_{10}$

Conclusion: Booth's Algorithm correctly computes $-6 \times +5 = -30$ while efficiently handling signed numbers and minimizing arithmetic operations.

Q4. Restoring and Non-Restoring Division Algorithm (13 ÷ 3)

Question:
A processor is required to perform division of two binary numbers (13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Problem Understanding (20%)

The processor must divide **13 by 3** using two binary division algorithms:

- **Restoring Division**
- **Non-Restoring Division**

The solution should:

- Show each step.
- Display intermediate remainders.
- Form the quotient.
- Compare both algorithms.

· Explain why non-restoring division is more efficient.

Method Selection (20%)

Dividend = 13

Binary:

13 = 1101_2

Divisor = 3

Binary:

3 = 0011_2

Registers used:

· A = Accumulator (Remainder)

· Q = Dividend / Quotient

· M = Divisor

Initially

A = 0000

Q = 1101

M = 0011

Solution Process (25%)

A) Restoring Division Algorithm

Step 1

Left Shift (AQ)
A = 0001

Q = 1010

Subtract M

0001

−0011

1110 (Negative)

Negative remainder

Restore by adding

M 1110

+0011

0001

Quotient bit = 0

Step 2

Shift

A = 0011

Q = 0100

Subtract

0011

−0011

0000

Positive

Quotient bit =

1 Q = 0101

Step 3

Shift

A = 0000

Q = 1010

Subtract

0000
−0011

1101 (Negative)

Restore

1101

+0011

0000

Quotient bit = 0

Step 4

Shift

A = 0001

Q = 0100

Subtract

0001

−0011

1110 (Negative)

Restore

1110

+0011

0001

Quotient bit = 0

Final Result

Quotient = $0100_2 = 4$

Remainder = $0001_2 =$

1 Therefore,

$13 \div 3 = 4$

Remainder = 1

B) Non-Restoring Division

Algorithm Initial

A = 0000

Q = 1101

M = 0011

Step 1

Shift

A = 0001

Q = 1010

Subtract M

0001

−0011

1110

Negative

Quotient bit = 0

(No restoration)

Step 2

Shift

A = 1101

Q = 0100

Since A is negative,

Add M

1101

+0011

0000

Quotient bit = 1

Step 3

Shift

A = 0000
Q = 1001

Subtract M

0000

−0011

1101

Negative

Quotient bit = 0

Step 4

Shift

Negative remainder

Add M

1110

+0011

0001

Positive

Quotient bit = 0

Final Result

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

Therefore,

$13 \div 3 = 4$

Remainder = 1

Accuracy of Calculation (20%)

Method Dividend Divisor Quotient Remainder Restoring

Division 13 3 **4 1** Non-Restoring Division 13 3 **4 1** Verification:

(Quotient × Divisor) + Remainder

$= (4 \times 3) + 1$

$= 12 + 1$

$= 13$ ✓

Comparison of Restoring and Non-Restoring Division

Feature Restoring Division Non-Restoring Division

Restoration step Required after negative remainder Not required

Arithmetic operations More Fewer

Hardware complexity Higher Lower

Execution speed Slower Faster

Efficiency Lower Higher

Modern processor usage Rare Commonly preferred

- Both restoring and non-restoring division algorithms correctly compute **13 ÷ 3**, producing a **quotient of 4** and a **remainder of 1**.
- Restoring division requires an additional restoration step whenever the remainder becomes negative, increasing the number of arithmetic operations.
- Non-restoring division avoids immediate restoration, reducing hardware operations and improving execution speed.
- Due to its lower complexity and higher efficiency, **non-restoring division is preferred in modern processors** for implementing binary division operations.

Q5. IEEE 754 Floating-Point Representation and Floating-Point Addition

(Answer based on the uploaded assessment question and written according to the assessment rubric.)

Question 5

A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given the decimal number -13.25, analyze how it is represented in both **single precision** and **double precision** formats, including **sign, exponent, and mantissa** fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like **normalization, rounding, and precision loss.**

1. Problem Understanding (20%)

IEEE 754 is the international standard used by computers to represent real (floating-point) numbers.
The decimal number -13.25 must be represented in:

- **Single Precision (32-bit)**
- **Double Precision (64-bit)**

Each representation consists of:

- **Sign bit**
- **Exponent**
- **Mantissa (Fraction)**

The floating-point addition process must also be explained with normalization, rounding, and precision considerations.

2. Method Selection (20%)

Use the IEEE 754 standard:

Single Precision (32-bit)

- Sign = 1 bit
- Exponent = 8 bits
- Mantissa = 23 bits
- Bias = **127**

Double Precision (64-bit)

- Sign = 1 bit
- Exponent = 11 bits
- Mantissa = 52 bits
- Bias = **1023**

3. Solution Process (25%)

Step 1: Convert Decimal Number to Binary

Given

Ignore the negative sign first.

Integer Part

13

Binary:

$$13 = (1101)_2$$

Fraction Part
0.25

Multiply by 2

0.25 × 2 = 0.50 → **0**

0.50 × 2 = 1.00 → **1**

Therefore,

$$0.25 = (0.01)_2$$

Hence,

$$13.25 = (1101.01)_2$$

Including sign,

$$-13.25 = (-1101.01)_2$$

Step 2: Normalize

Move binary point after first 1.

$$1101.01 = 1.10101 \times 2^3$$

Therefore,

Normalized form is

$$1.10101 \times 2^3$$

Single Precision Representation (32-bit)

Step 3: Sign Bit

Since number is negative,

Sign bit

$$□□ = 1$$

Step 4: Exponent

Actual exponent

$$□□ = 3$$

Bias

$$127$$

Stored exponent

$$3 + 127 = 130$$

130 in binary

$$130 = (10000010)_2$$

Exponent field

$$10000010$$

Step 5: Mantissa

Normalized number

$$1.10101$$

Leading 1 is omitted.

Mantissa becomes

$$10101000000000000000000$$

(23 bits)

Final IEEE 754 Single Precision

Field Bits

Sign 1

Exponent 10000010

Mantissa 10101000000000000000000

Complete 32-bit representation:

1 10000010 10101000000000000000000

Double Precision Representation (64-bit)

Sign Bit

Exponent

Bias
1023

Stored exponent

$$1023 + 3 = 1026$$

1026 in binary

$$10000000010$$

Mantissa

Fraction after leading 1

$$10101$$

Fill remaining bits with zeros until 52 bits.

Mantissa

1010100000000000000000000000000000000000000000000000

Final IEEE 754 Double Precision

Field Bits

Sign 1

Exponent 10000000010

Mantissa 1010100000000000000000000000000000000000000000000000

Complete 64-bit representation:

1 10000000010 1010100000000000000000000000000000000000000000000000

4. Floating-Point Addition Process

Example:

$$13.25 + 2.5$$

Step 1

Convert both numbers into IEEE 754 format.

Step 2

Compare exponents.

13.25

$$1.10101 \times 2^3$$

2.5

$$1.01 \times 2^1$$

Step 3

Align Exponents

Increase exponent of second number to 3.

Shift mantissa right.

$$1.01 \times 2^1$$

becomes

$$0.0101 \times 2^3$$

Now both exponents are equal.

Step 4

Add Mantissas

1.10101

+0.01010

1.11111

Result

$$1.11111 \times 2^3$$

Step 5

Normalize

If result is not in normalized form, shift binary point until leading digit becomes 1.

Step 6

Round

If mantissa exceeds available bits

- Guard bit
- Round bit
- Sticky bit

are used to round to the nearest representable value.

Step 7

Store Result

Store:

- Sign
- Exponent
- Mantissa

according to IEEE 754 format.

5. Accuracy of Calculation (20%)

Single Precision

- Sign = **1**
- Exponent = **10000010**
- Mantissa = **10101000000000000000000**

Double Precision

- Sign = **1**
- Exponent = **10000000010**
- Mantissa = **1010100000000000000000000000000000000000000000000000**

These fields correctly represent -13.25 using IEEE 754 format.

6. Interpretation of Results (15%)

- IEEE 754 provides a standard method to represent floating-point numbers across computer systems.
- **Single precision (32-bit)** uses less memory and is suitable for general-purpose applications but offers lower precision.
- **Double precision (64-bit)** provides higher accuracy and is preferred for scientific, engineering, and financial computations.
- During floating-point addition, exponents must first be aligned, after which mantissas are added. The result is then normalized and rounded before being stored.
- Precision loss may occur because many decimal fractions cannot be represented exactly in binary, making IEEE 754 rounding essential for maintaining consistent and reliable numerical results.